

Lab 12 & Lab 13

Vishal Baraiya

Section 1: PyTorch

Import Torch

```
In [1]: import torch
import torch.nn as nn
import torch.optim as optim
```

```
In [2]: pip install torch
```

Requirement already satisfied: torch in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (2.10.0)Note: you may need to restart the kernel to use updated packages.

Requirement already satisfied: filelock in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (3.18.0)
Requirement already satisfied: typing-extensions>=4.10.0 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (4.14.1)
Requirement already satisfied: sympy>=1.13.3 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (1.14.0)
Requirement already satisfied: networkx>=2.5.1 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (3.4.2)
Requirement already satisfied: jinja2 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (3.1.6)
Requirement already satisfied: fsspec>=0.8.5 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (2025.5.1)
Requirement already satisfied: setuptools in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from torch) (80.9.0)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from sympy>=1.13.3->torch) (1.3.0)
Requirement already satisfied: MarkupSafe>=2.0 in C:\Users\ASUS\AppData\Local\Programs\Python\Python313\Lib\site-packages (from jinja2->torch) (3.0.2)

Task 1: GPU/Device Agnostic Code

Goal: Write code that runs on CPU, CUDA, or MPS (Mac) automatically.

```
In [3]: device = 'cuda' if torch.cuda.is_available() else 'mps' if torch.backends.mps.is_av
print(f'Using device: {device}')

sample_input = torch.randn(1, 10).to(device)
type(sample_input)
```

Using device: cpu

Out[3]: torch.Tensor

```
In [4]: sample_input
```

```
Out[4]: tensor([[ -0.8814,  0.1599,  0.9664, -0.0369, -1.2109,  0.3326,  0.3727, -0.8851,
                  1.4217,  2.0016]])
```

Section 2: MNIST Project

Step 1: What is MNIST & Downloading Data

Concept: MNIST is the "Hello World" of Machine Learning. It contains 70,000 images of handwritten digits (0-9).

The Goal: Teach the computer to look at a grid of pixels and say "That is a 7".

The Data: Each image is grayscale and exactly 28×28

```
In [5]: from torchvision import datasets
        from torchvision.transforms import ToTensor
```

```
In [6]: training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)

# transform=ToTensor() converts the image (0-255) to a Torch Tensor (0.0-1.0)
```

```
In [9]: len(test_data)
```

```
Out[9]: 10000
```

```
In [10]: img, label = training_data[1000]
```

```
In [11]: img.shape
```

```
Out[11]: torch.Size([1, 28, 28])
```

```
In [12]: img
```

```
Out[12]: tensor([[[[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000, 0.3804, 0.3294, 0.3804, 0.4314, 0.4235,
0.3412, 0.3804, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0745, 1.0000, 0.7804, 0.7176, 0.5569, 0.6314,
0.7843, 0.8353, 0.1255, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.2824, 0.6824, 0.6784, 0.6196, 0.5137, 0.5176,
0.6902, 0.6902, 0.2745, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.5569, 0.7059, 0.6392, 0.4706, 0.7059, 0.4941,
0.5451, 0.6588, 0.4784, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.6118, 0.7059, 0.6353, 0.6706, 0.6667, 0.7176,
0.6392, 0.6627, 0.5647, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.6235, 0.6941, 0.6392, 0.6824, 0.6510, 0.6784,
0.6667, 0.6392, 0.5804, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.6275, 0.7137, 0.6275, 0.6588, 0.6863, 0.6549,
0.6157, 0.6588, 0.5451, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.5608, 0.7255, 0.6314, 0.6706, 0.7216, 0.6902,
0.6275, 0.6863, 0.5137, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.5608, 0.7373, 0.6078, 0.6745, 0.7137, 0.7059,
0.6314, 0.7137, 0.4980, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.4980, 0.6667, 0.6588, 0.6863, 0.8275, 0.7490,
0.6392, 0.7216, 0.4431, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.3059, 0.5176, 0.6863, 0.6941, 0.5059, 0.7961,
0.6314, 0.7333, 0.4000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.3569, 0.6510, 0.6392, 0.7451, 0.2471, 0.7961,
0.6235, 0.7412, 0.3686, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.3843, 0.4941, 0.6510, 0.7529, 0.2549, 0.7765,
0.6549, 0.7569, 0.3294, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.3529, 0.4980, 0.6667, 0.7765, 0.2392, 0.7843,
0.6824, 0.7765, 0.2784, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000]]]])
```

```
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.3647, 0.5529, 0.5922, 0.8118, 0.2157, 0.7686,
0.6706, 0.7686, 0.2275, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.3451, 0.5373, 0.5333, 0.8235, 0.2118, 0.7765,
0.6784, 0.7804, 0.1804, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.2824, 0.4902, 0.5137, 0.8392, 0.2157, 0.7804,
0.6902, 0.7843, 0.1490, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.2196, 0.5725, 0.4941, 0.8235, 0.2196, 0.7843,
0.6863, 0.7843, 0.1451, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.1569, 0.9294, 0.6706, 0.7647, 0.2510, 0.7804,
0.6824, 0.7961, 0.1412, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0902, 0.7451, 0.6588, 0.7529, 0.2745, 0.7686,
0.6706, 0.7961, 0.1176, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0588, 0.7373, 0.6706, 0.7686, 0.2745, 0.7765,
0.6784, 0.7922, 0.0824, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0431, 0.7333, 0.6745, 0.7725, 0.2784, 0.7804,
0.6824, 0.7961, 0.0588, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0039, 0.7176, 0.6863, 0.7765, 0.2863, 0.7647,
0.6824, 0.8039, 0.0510, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.7294, 0.6941, 0.7843, 0.3059, 0.7490,
0.6863, 0.6706, 0.0314, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.7098, 0.6941, 0.7765, 0.3059, 0.7412,
0.6902, 0.6510, 0.0157, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.6980, 0.7137, 0.7137, 0.4196, 0.7216,
0.6863, 0.7569, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.6784, 0.7843, 0.8078, 0.3294, 0.7686,
0.7686, 0.5216, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000],
[0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.2667, 0.8118, 0.7137, 0.0000, 0.6863,
0.7647, 0.1333, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000, 0.0000,
0.0000, 0.0000, 0.0000, 0.0000]]])
```

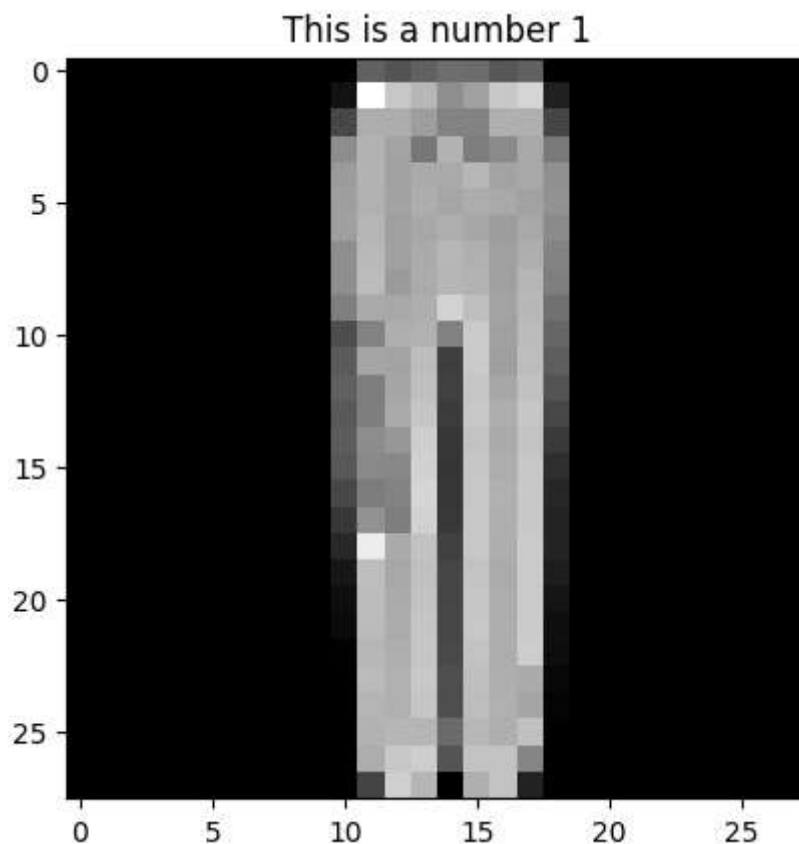
```
In [13]: img.squeeze().shape
```

```
Out[13]: torch.Size([28, 28])
```

Display one Image in Matplotlib

```
In [14]: import matplotlib.pyplot as plt
```

```
In [15]: plt.imshow(img.squeeze(), cmap="gray")  
plt.title(f"This is a number {label}")  
plt.show()
```



Calculation for the Input Layer

Concept: We are building a Linear (Feed-Forward) Network,

A Linear Layer consists of neurons in a single vertical line.

Our image is a square grid (28×28).

The Division: We must "cut" the image row by row and stack them into one long line.

The Calculation:

Height×Width=Total Input Features $28 \times 28 = 784$ So, our Input Layer must have 784 neurons.

```
In [16]: img, label = training_data[1]
```

```
In [17]: img.squeeze().shape
```

```
Out[17]: torch.Size([28, 28])
```

The Architecture (1 Input, 1 Hidden, 1 Output)

The Concept: We will build the simplest standard network.

Input Layer (784): Receives the pixels.

Hidden Layer (128): The "brain" that learns shapes (loops, lines). We pick 128 because it's enough to learn but not too big.

Output Layer (10): The final decision. We have 10 digits (0-9), so we need 10 output scores.

```
In [18]: import torch.nn as nn

class SimpleNet(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()

        self.layers = nn.Sequential(
            nn.Linear(784, 128),
            nn.ReLU(), # Activation
            nn.Linear(128, 10)
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.layers(x)
        return logits

model = SimpleNet()
print(model)
```

```
SimpleNet(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (layers): Sequential(
    (0): Linear(in_features=784, out_features=128, bias=True)
    (1): ReLU()
    (2): Linear(in_features=128, out_features=10, bias=True)
  )
)
```

Understanding Batch Size (The Stack)

The Concept: Think of the model like a teacher grading exams.

Batch Size = 1: The teacher grades 1 exam, updates the grade book, then picks up the next exam. (Too slow).

Batch Size = 64: The teacher picks up a stack of 64 exams, grades them all at once, and updates the grade book one time for the whole stack. (Much faster).

We use DataLoader to create these "stacks" for us.

```
In [19]: from torch.utils.data import DataLoader

# Create stacks
train_loader = DataLoader(training_data, batch_size=64, shuffle=True)
test_loader = DataLoader(test_data, batch_size=64, shuffle=True)

print(len(train_loader), "--", len(test_loader))
```

938 -- 157

The Training Loop

```
In [20]: optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
loss_fn = nn.CrossEntropyLoss()
```

```
In [21]: def train():
    model.train()

    # Epoch Loop - Train for multiple epochs
    num_epochs = 5

    for epoch in range(num_epochs):
        print(f"\n--- Epoch {epoch + 1}/{num_epochs} ---")

        for batch_No, (data, target) in enumerate(train_loader):

            output = model(data)

            loss = loss_fn(output, target)

            # Backpropagation
            optimizer.zero_grad() # Clear old calculations
            loss.backward()        # Calculate gradients
            optimizer.step()       # Update weights

            if batch_No % 100 == 0:
                print(f"Batch {batch_No}: Loss = {loss.item():.4f}")

        print(f"Epoch {epoch + 1} completed!")

    # Run training
```


In []:

In [22]: `train()`

--- Epoch 1/5 ---

Batch 0: Loss = 2.3200
Batch 100: Loss = 0.7404
Batch 200: Loss = 0.7883
Batch 300: Loss = 0.4096
Batch 400: Loss = 0.3718
Batch 500: Loss = 0.5079
Batch 600: Loss = 0.3543
Batch 700: Loss = 0.4186
Batch 800: Loss = 0.5116
Batch 900: Loss = 0.5107
Epoch 1 completed!

--- Epoch 2/5 ---

Batch 0: Loss = 0.2942
Batch 100: Loss = 0.4656
Batch 200: Loss = 0.3515
Batch 300: Loss = 0.5046
Batch 400: Loss = 0.5124
Batch 500: Loss = 0.3701
Batch 600: Loss = 0.3881
Batch 700: Loss = 0.3967
Batch 800: Loss = 0.2345
Batch 900: Loss = 0.3681
Epoch 2 completed!

--- Epoch 3/5 ---

Batch 0: Loss = 0.3372
Batch 100: Loss = 0.4262
Batch 200: Loss = 0.2916
Batch 300: Loss = 0.3294
Batch 400: Loss = 0.2854
Batch 500: Loss = 0.4834
Batch 600: Loss = 0.4272
Batch 700: Loss = 0.3219
Batch 800: Loss = 0.5026
Batch 900: Loss = 0.2760
Epoch 3 completed!

--- Epoch 4/5 ---

Batch 0: Loss = 0.4107
Batch 100: Loss = 0.3161
Batch 200: Loss = 0.3582
Batch 300: Loss = 0.3555
Batch 400: Loss = 0.3803
Batch 500: Loss = 0.2860
Batch 600: Loss = 0.3142
Batch 700: Loss = 0.2456
Batch 800: Loss = 0.2104
Batch 900: Loss = 0.3854
Epoch 4 completed!

--- Epoch 5/5 ---

Batch 0: Loss = 0.2828
Batch 100: Loss = 0.2624
Batch 200: Loss = 0.2076

```
Batch 300: Loss = 0.2915
Batch 400: Loss = 0.2611
Batch 500: Loss = 0.2170
Batch 600: Loss = 0.3404
Batch 700: Loss = 0.3337
Batch 800: Loss = 0.5421
Batch 900: Loss = 0.3459
Epoch 5 completed!
```

```
In [23]: def test():
          model.eval()
          correct = 0

          with torch.no_grad():
              for data, target in test_loader:
                  output = model(data)

                  prediction = output.argmax(dim=1)

                  correct += (prediction == target).sum().item()

          accuracy = correct / len(test_loader.dataset)
          print(f"Test Accuracy: {accuracy:.1%}")

          test()
```

Test Accuracy: 87.2%

In []: